

Pontificia Universidad Católica del Perú
Maestría en Ingeniería de Control y Automatización
Examen Final 2025-1 — Desarrollo

Curso: Redes Neuronales Aplicadas al Control

Guía de estudio elaborada a partir de las notas del curso (Clases 7, 9, 10, 11, 12 y 13)

Pregunta 1 (4 ptos) — Control lineal–difuso del robot truck–tráiler para coordenadas iniciales y alejadas de $y^* = 0$

Se ha diseñado un controlador para un robot móvil tipo truck–tráiler de dos cuerpos articulados basado en la integración de un controlador difuso con un controlador lineal diseñado según el método de linealización por aproximación, para alcanzar la posición deseada $y^ = 0$, $\theta_2^* = 0$, $\theta_{12}^* = 0$, $\delta^* = 0$. El controlador trabaja bien para valores iniciales de la coordenada y cercanos a 0. Explicar el proceso de diseño del controlador para que el robot alcance la posición deseada desde coordenadas iniciales y alejadas de $y^* = 0$.*

1.1 Revisión del diseño base (Clase 13)

Paso 1 — Modelo cinemático. El robot truck–tráiler se modela como dos barras articuladas (truck de longitud L_1 , tráiler de longitud L_2), moviéndose en retroceso a velocidad constante v , sin deslizamiento de ruedas. Con $\theta_{12} = \theta_1 - \theta_2$ (ángulo truck–tráiler) las ecuaciones de movimiento son:

$$\dot{y} = v \cos \theta_{12} \sin \theta_2 \quad (1)$$

$$\dot{\theta}_2 = -\frac{v}{L_2} \sin \theta_{12} \quad (2)$$

$$\dot{\theta}_{12} = \frac{v}{L_2} \sin \theta_{12} - \frac{v}{L_1} \tan \delta \quad (3)$$

Se define el vector de estado $\mathbf{x} = [y \ \theta_2 \ \theta_{12}]^T$ y la entrada $u = \tan \delta$. La coordenada x no se incluye en el estado: como v es constante, al converger a la recta $y = 0$ se cumple $\dot{x} = v$, es decir x se comporta como el tiempo (variable independiente), y la estrategia de control es: *primero* alcanzar la recta $y^* = 0$ con $\theta_2^* = 0$, $\theta_{12}^* = 0$ y *después* avanzar en línea recta hasta la posición final, pasando entre los obstáculos.

Paso 2 — Linealización por aproximación. Alrededor del punto deseado ($y^* = 0$, $\theta_2^* = 0$, $\theta_{12}^* = 0$, $\delta^* = 0$) se aproxima $\sin \theta \approx \theta$, $\cos \theta \approx 1$:

$$\begin{bmatrix} \dot{y} \\ \dot{\theta}_2 \\ \dot{\theta}_{12} \end{bmatrix} = \begin{bmatrix} 0 & v & 0 \\ 0 & 0 & -v/L_2 \\ 0 & 0 & v/L_2 \end{bmatrix} \begin{bmatrix} y \\ \theta_2 \\ \theta_{12} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ -v/L_1 \end{bmatrix} \tan \delta \quad (4)$$

Paso 3 — Controlador lineal. Para el modelo (4) se diseña una ley de realimentación completa de estados

$$\tan \delta = -k_1 y - k_2 \theta_2 - k_3 \theta_{12} \quad (5)$$

eligiendo k_1, k_2, k_3 por asignación de polos (o LQR) de modo que el lazo cerrado lineal sea estable.

Paso 4 — Integración difusa en θ_{12} (evitar jack–knife). La ley (5) sólo es válida cerca de $\theta_{12} = 0$; para θ_{12} cercanos a $\pm 90^\circ$ (posición jack–knife) no garantiza estabilidad. El rango de θ_{12} se particiona en tres valores lingüísticos (*Negative Big*, *Zero*, *Positive Big*), se diseña un controlador simple para cada partición y se integran mediante lógica difusa. De la ecuación (3) se observa la relación inversa entre $\dot{\theta}_{12}$ y $\tan \delta$, lo que da las reglas:

IF $\theta_{12} = \text{Positive Big}$ **THEN** $\delta = \delta_{\max}$ (positivo)
IF $\theta_{12} = \text{Zero}$ **THEN** $\delta = \text{ley lineal (5)}$
IF $\theta_{12} = \text{Negative Big}$ **THEN** $\delta = \delta_{\min}$ (negativo)

La salida del controlador integrado es la **suma ponderada** de las salidas de los tres controladores, con pesos dados por los valores de pertenencia de θ_{12} a cada partición:

$$\delta = \frac{\mu_{NB}(\theta_{12}) \delta_{\min} + \mu_{ZE}(\theta_{12}) \delta_{lin} + \mu_{PB}(\theta_{12}) \delta_{\max}}{\mu_{NB}(\theta_{12}) + \mu_{ZE}(\theta_{12}) + \mu_{PB}(\theta_{12})} \quad (6)$$

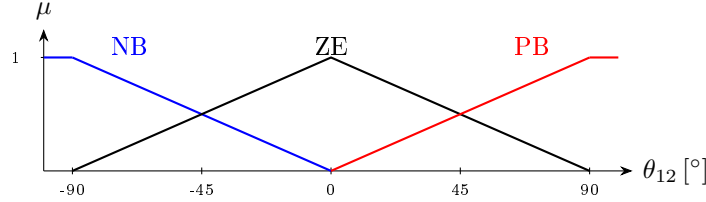


Figura 1: Particiones y funciones de pertenencia del ángulo truck-tráiler θ_{12} (traslape entre particiones contiguas; ancho de la partición ZE ajustable: una ZE más ancha da convergencia más rápida a costa de θ_{12} y δ mayores).

1.2 ¿Por qué el diseño base sólo funciona con y cercano a 0?

- La linealización sin $\theta_2 \approx \theta_2$ sólo vale para θ_2 pequeños. Pero para acercarse a la recta $y = 0$ desde un y lejano, el robot *necesita* inclinaciones θ_2 grandes (debe “apuntar” hacia la recta): la maniobra correcta sale de la región de validez del modelo linealizado.
- Con y grande, el término $-k_1 y$ de la ley (5) produce valores de $\tan \delta$ enormes: el timón se satura ($\delta \in [-30^\circ, 30^\circ]$), el lazo deja de comportarse como el lazo lineal diseñado y pueden inducirse posiciones jack-knife.
- La ley lineal intenta lograr *simultáneamente* $y \rightarrow 0$ y $\theta_2 \rightarrow 0$, lo que es geoméricamente imposible desde lejos: primero hay que reorientar el vehículo hacia la recta y sólo al final alinear $\theta_2 \rightarrow 0$.

1.3 Proceso de diseño extendido para y alejados de $y^* = 0$

La idea del curso se mantiene: *diseñar controladores simples válidos en regiones (particiones) del espacio de estado e integrarlos mediante lógica difusa como suma ponderada por las pertenencias*. Ahora la partición se hace también sobre la coordenada y .

Paso 1 — Consigna intermedia de orientación. Cuando $|y|$ es grande, el objetivo instantáneo ya no es $\theta_2^* = 0$ sino *aproximarse a la recta* con un ángulo de aproximación grande. Se define la consigna de inclinación deseada del tráiler como función saturada de y :

$$\theta_2^*(y) = -\bar{\theta}_2 \text{sat}\left(\frac{y}{y_a}\right), \quad \text{sat}(s) = \begin{cases} 1 & s > 1 \\ s & |s| \leq 1 \\ -1 & s < -1 \end{cases} \quad (7)$$

donde $\bar{\theta}_2$ es el ángulo máximo de aproximación (por ejemplo 60° – 90° , con el signo elegido según el sentido de avance v de modo que el movimiento reduzca $|y|$) y y_a define el ancho de la zona de transición. Lejos de la recta el robot se aproxima con inclinación $\pm \bar{\theta}_2$ casi constante; cerca de la recta la consigna tiende suavemente a 0.

Paso 2 — Ley lineal con consignas variables. Se usa la versión de la ley lineal para posiciones deseadas arbitrarias (ecuación (13) del paper de la Clase 13), con $\theta_{12}^* = 0$ y $\delta^* = 0$:

$$\tan \delta = -k_2 (\theta_2 - \theta_2^*(y)) - k_3 \theta_{12} \quad (8)$$

Verificación de consistencia: cerca de la recta, $\text{sat}(y/y_a) \approx y/y_a$ y la ley (8) se reduce a $\tan \delta \approx -(k_2 \bar{\theta}_2 / y_a) y - k_2 \theta_2 - k_3 \theta_{12}$, es decir, se *recupera exactamente* la ley lineal original (5) con $k_1 \equiv k_2 \bar{\theta}_2 / y_a$. La transición es suave, sin conmutaciones bruscas.

Paso 3 — Formulación difusa equivalente. Alternativamente (y de forma totalmente coherente con la metodología del curso), se particiona y en tres valores lingüísticos $\{NG, ZE, PG\}$ y se asigna un controlador simple a cada partición:

IF $y = PG$ (muy arriba) **THEN** $\delta = \text{ley (8) con } \theta_2^* = -\bar{\theta}_2$ (bajar hacia la recta)
IF $y = ZE$ (cerca de la recta) **THEN** $\delta = \text{ley lineal original (5)}$
IF $y = NG$ (muy abajo) **THEN** $\delta = \text{ley (8) con } \theta_2^* = +\bar{\theta}_2$ (subir hacia la recta)

y la salida total es la suma ponderada por las pertenencias $\mu_{NG}(y), \mu_{ZE}(y), \mu_{PG}(y)$, igual que en (6).

Paso 4 — Prioridad anti jack-knife. Las reglas del diseño base sobre θ_{12} (partición $NB/ZE/PB$) se mantienen y *dominan* sobre las reglas en y : si θ_{12} entra en PB o NB , la salida se satura en $\delta_{\max}$ o $\delta_{\min}$ para llevar θ_{12} hacia cero, independientemente de lo que pidan las reglas de posicionamiento. Esto garantiza que las maniobras amplias requeridas desde y lejano no produzcan jack-knife.

Paso 5 — Verificación por simulación. Se simula el lazo cerrado desde múltiples posiciones iniciales (distintos $y_0, \theta_{2,0}, \theta_{12,0}$), verificando: (i) convergencia asintótica a $y^* = 0, \theta_2^* = 0, \theta_{12}^* = 0$; (ii) δ acotado en $[\delta_{\min}, \delta_{\max}] = [-30^\circ, +30^\circ]$; (iii) $|\theta_{12}|$ lejos de 90° en todo momento; (iv) no colisión con los obstáculos. Con los resultados se ajustan $\bar{\theta}_2, y_a$ y el ancho de las particiones.

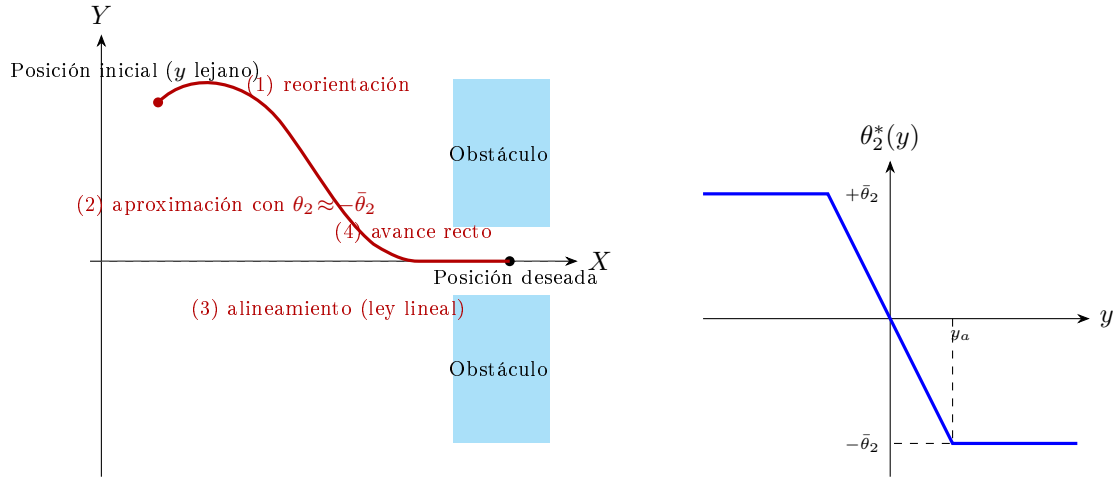


Figura 2: Izquierda: fases de la trayectoria con el controlador extendido. Derecha: consigna de orientación saturada $\theta_2^*(y)$; en la zona $|y| < y_a$ se recupera la ley lineal original.

Pregunta 2 (4 ptos) — Controlador difuso para el ángulo θ_2 de un motor CC con reductor

Diseñar un sistema difuso para controlar el ángulo de giro θ_2 del motor CC con reductor. Sistema: $\dot{\mathbf{x}} = A\mathbf{x} + Bu$, $\mathbf{x} = [\theta_2 \ \dot{\theta}_2 \ i]^T$. Voltaje $\in [-24, 24]$ V; $\theta_2 \in [-45^\circ, 45^\circ]$; $\dot{\theta}_2 \in [-10, 10]$ °/s. Tres particiones para las entradas y tres para la salida. Voltaje positivo $\Rightarrow$ giro antihorario (positivo). Objetivo: $\theta_2 \rightarrow 0$ desde cualquier posición inicial. Explicar todo el proceso de diseño, el cálculo de la salida para valores arbitrarios de θ_2 y $\dot{\theta}_2$, y representar el controlador mediante una red neuro-difusa.

2.1 Entradas y salida del controlador (proceso de diseño, Clase 11)

Aunque el sistema tiene tres variables de estado, el controlador difuso usa como entradas las dos variables dominantes del objetivo de control: la **posición angular** θ_2 y la **velocidad angular** $\dot{\theta}_2$ (la corriente i tiene dinámica eléctrica rápida y no se necesita como entrada del controlador). La velocidad es indispensable: sin ella el controlador no puede “frenar” a tiempo y el eje oscilaría alrededor de $\theta_2^* = 0$. La salida es el voltaje V aplicado al motor.

$$\theta_2^* = 0, \quad \dot{\theta}_2^* = 0, \quad V^* = 0$$

Los valores deseados quedan en el **punto medio** de cada rango, como exige la metodología del curso.

2.2 Rangos de variación

$$-45^\circ \leq \theta_2 \leq 45^\circ, \quad -10 \text{ °/s} \leq \dot{\theta}_2 \leq 10 \text{ °/s}, \quad -24 \text{ V} \leq V \leq 24 \text{ V}$$

(El rango de $\dot{\theta}_2$ se determina experimentalmente aplicando el voltaje máximo y midiendo la velocidad alcanzada; si fuera necesario se da un margen adicional del 20–30%. Al voltaje también podría dársele margen, p.ej. ± 30 V, para garantizar que la defuzzificación pueda alcanzar el voltaje máximo físico; aquí mantenemos ± 24 V del enunciado.)

2.3 Particiones, valores lingüísticos y funciones de pertenencia

Cada variable se particiona en un número **impar** (3) de particiones **simétricas**, con **traslape** entre particiones contiguas y suma de pertenencias igual a 1:

$$\theta_2 : \{NE, ZE, PO\}, \quad \dot{\theta}_2 : \{NE, ZE, PO\}, \quad V : \{NE, ZE, PO\}$$

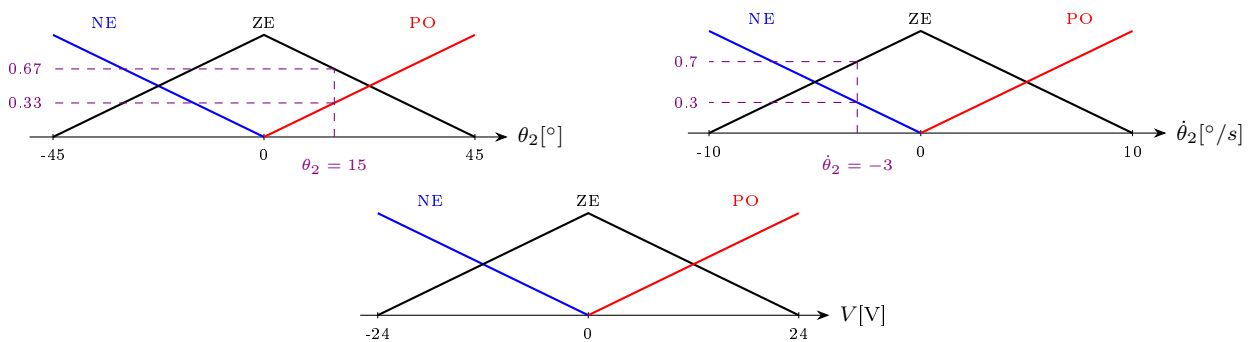


Figura 3: Funciones de pertenencia triangulares de θ_2 , $\dot{\theta}_2$ y V (traslape, simetría, suma = 1), con la fuzzificación del ejemplo $\theta_2 = +15^\circ$, $\dot{\theta}_2 = -3$ °/s.

2.4 Base de reglas (9 reglas)

Física del sistema: voltaje positivo $\Rightarrow$ el eje 2 gira en sentido antihorario ($\dot{\theta}_2$ aumenta). Para llevar $\theta_2 \rightarrow 0$: si $\theta_2 > 0$ se necesita velocidad negativa (voltaje negativo); si $\theta_2 < 0$, voltaje positivo; y la velocidad se usa para frenar (amortiguar) al acercarse. Para cada combinación de valores lingüísticos de las entradas se formula una regla con la experiencia de un operador humano ($3 \times 3 = 9$ reglas):

θ_2	$\dot{\theta}_2$		
	NE	ZE	PO
NE	PO	PO	ZE
ZE	PO	ZE	NE
PO	ZE	NE	NE

Tabla 1: Base de reglas: valor lingüístico del voltaje V .

Justificación de reglas representativas:

- **IF $\theta_2 = PO$ AND $\dot{\theta}_2 = PO$ THEN $V = NE$:** el eje está adelantado *y alejándose*; hay que frenar y regresarlo con voltaje negativo.
- **IF $\theta_2 = PO$ AND $\dot{\theta}_2 = NE$ THEN $V = ZE$:** el eje está adelantado pero *ya regresa* solo; no se aplica voltaje para no sobrepasar el objetivo.
- **IF $\theta_2 = ZE$ AND $\dot{\theta}_2 = PO$ THEN $V = NE$:** en el objetivo pero moviéndose: frenar.
- **IF $\theta_2 = ZE$ AND $\dot{\theta}_2 = ZE$ THEN $V = ZE$:** objetivo logrado, no actuar.

(Las reglas con $\theta_2 = NE$ son simétricas.) La base de reglas es antisimétrica respecto al centro, como corresponde a un problema de estabilización simétrico: es un “PD difuso”.

2.5 Cálculo de la salida: inferencia MIN-MAX (Mamdani) con ejemplo numérico

Valores arbitrarios elegidos: $\theta_2 = +15^\circ$, $\dot{\theta}_2 = -3^\circ/s$

(i) **Fuzzificación** (ver Figura 3):

$$\mu_{ZE}(\theta_2) = 1 - \frac{15}{45} = 0.67, \quad \mu_{PO}(\theta_2) = \frac{15}{45} = 0.33, \quad \mu_{NE}(\dot{\theta}_2) = \frac{3}{10} = 0.30, \quad \mu_{ZE}(\dot{\theta}_2) = 1 - \frac{3}{10} = 0.70$$

(las demás pertenencias son cero).

(ii) **Reglas activas y operador AND = mínimo** (se activan $2 \times 2 = 4$ reglas):

Regla	Antecedente	Consecuente	Grado de activación
R1	$\theta_2 = ZE$ AND $\dot{\theta}_2 = NE$	$V = PO$	$\min(0.67, 0.30) = 0.30$
R2	$\theta_2 = ZE$ AND $\dot{\theta}_2 = ZE$	$V = ZE$	$\min(0.67, 0.70) = 0.67$
R3	$\theta_2 = PO$ AND $\dot{\theta}_2 = NE$	$V = ZE$	$\min(0.33, 0.30) = 0.30$
R4	$\theta_2 = PO$ AND $\dot{\theta}_2 = ZE$	$V = NE$	$\min(0.33, 0.70) = 0.33$

(iii) **Agregación (máximo por consecuente)**: la partición PO queda recortada a 0.30; la partición ZE a $\max(0.67, 0.30) = 0.67$; y la partición NE a 0.33.

(iv) **Defuzzificación**.

Método de Sugeno (valores representativos $NE = -24$, $ZE = 0$, $PO = +24$):

$$V = \frac{0.30(+24) + 0.67(0) + 0.30(0) + 0.33(-24)}{0.30 + 0.67 + 0.30 + 0.33} = \frac{7.2 - 8.0}{1.60} = \boxed{-0.5 \text{ V}}$$

Método del centro de gravedad (Mandami): se toma el centroide de del área agregada (triángulos recortados, envolvente por máximo), obteniéndose $V \approx -0.21$ V.

Interpretación física: el eje está adelantado ($+15^\circ$) pero ya está regresando ($-3^\circ/s$); el controlador aplica un voltaje pequeño y negativo: deja que el eje regrese casi solo, empujándolo levemente hacia $\theta_2 = 0$ sin provocar sobrepaso. El resultado es coherente.

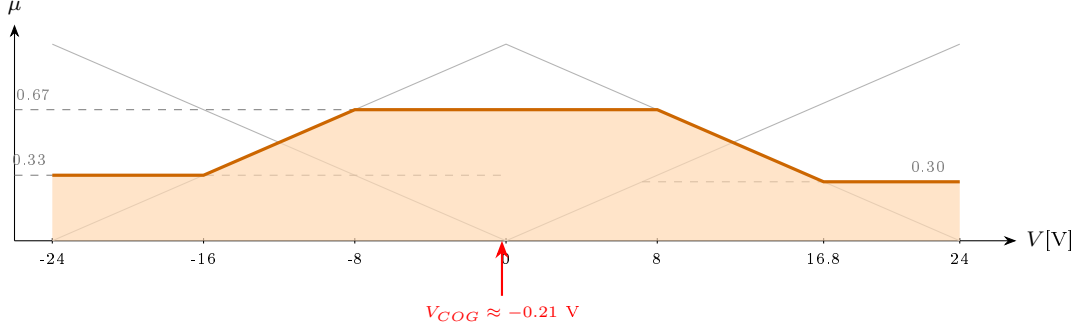


Figura 4: Salidas recortadas por los grados de activación, área agregada (máximo) y centro de gravedad.

2.6 Representación mediante una red neuro-difusa (Clase 12, red Tipo 1)

El controlador difuso se representa con una red neuronal cuya estructura reproduce las etapas de la lógica difusa:

- **Capa 1 (entradas):** valores numéricos θ_2 y $\dot{\theta}_2$.
- **Capa 2 (fuzzificación):** 6 neuronas, una por función de pertenencia (NE, ZE, PO de cada entrada). Cada neurona implementa una función de pertenencia parametrizada (p.ej. triangular o sigmoidea con *centro* c y *pendiente* a).
- **Capa 3 (reglas):** 9 neuronas; la regla R_i calcula su grado de activación μ_i con el operador AND implementado como **producto** (o mínimo) de las dos pertenencias de su antecedente.
- **Capa 4 (normalización):** calcula el factor $\Sigma = \sum_i \mu_i$ y normaliza $\bar{\mu}_i = \mu_i / \Sigma$.
- **Capa 5 (defuzzificación / salida):** suma ponderada con los valores representativos f_i del consecuente de cada regla ($f_i \in \{-24, 0, +24\}$ V según la base de reglas):

$$V = \frac{\sum_{i=1}^9 \mu_i f_i}{\sum_{i=1}^9 \mu_i} = \sum_{i=1}^9 \bar{\mu}_i f_i$$

Inicialmente la red neuro-difusa trabaja *exactamente igual* que el controlador difuso diseñado. Su ventaja es que puede **entrenarse** (p.ej. con el algoritmo DBP de la Pregunta 3) para ajustar los valores representativos f_i del consecuente o los parámetros (centros, pendientes) de las funciones de pertenencia, considerando el factor de normalización en el cálculo de las derivadas.

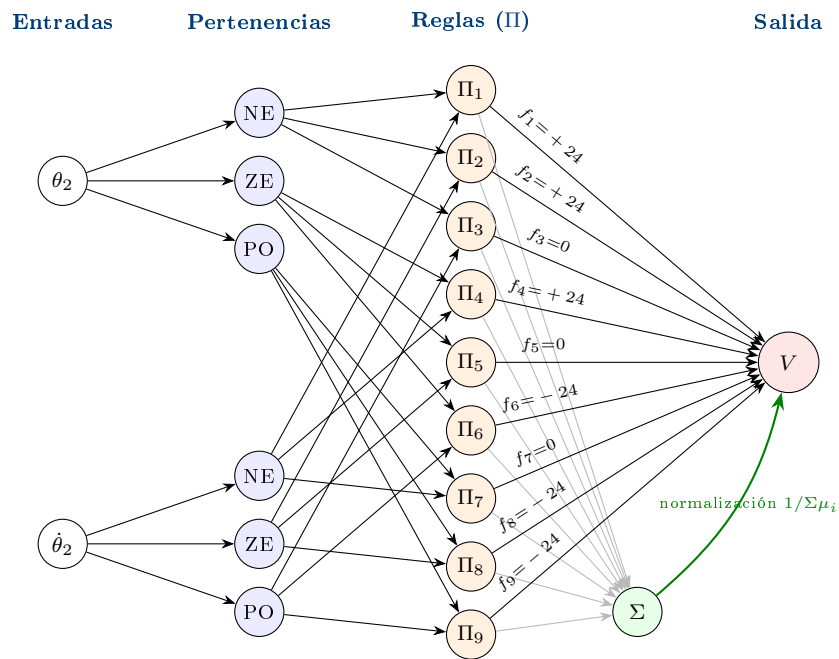


Figura 5: Red neuro-difusa Tipo 1 que representa al controlador difuso del motor CC. Los pesos de la última capa son los valores representativos f_i de cada regla.

Pregunta 3 (4 ptos) — Derivadas totales para el entrenamiento de neuro-controladores con el algoritmo DBP

Explicar el proceso de cálculo de derivadas totales para el entrenamiento de neuro-controladores con el algoritmo DBP. Definir la función de costo y derivar con detalle la ecuación recursiva. Explicar qué términos se calculan con el modelo del sistema y qué términos con el neuro-controlador. Determinar las matrices de la ecuación recursiva para el robot móvil tipo carro con ecuaciones en tiempo continuo $\dot{x} = v \cos \phi$, $\dot{y} = v \sin \phi$, $\dot{\phi} = -\frac{v}{L}u$.

3.1 Planteamiento del problema de entrenamiento dinámico

El lazo cerrado integra el neuro-controlador (red estática de una capa oculta, pesos v , w y parámetros de neurona c , a) y el modelo del sistema:

$$u_k = N(\mathbf{x}_k; v, c, a, w), \quad \mathbf{x}_{k+1} = \Phi(\mathbf{x}_k, u_k)$$

La salida del sistema en el paso k se convierte en entrada del controlador (y del modelo) en el paso $k+1$: aunque el controlador es una red estática, *el lazo cerrado es un sistema dinámico*. Al desplegar el lazo en el tiempo (pasos $1, 2, \dots, N$), la misma red con los mismos pesos aparece en cada paso, y $\mathbf{x}_k$ depende de los pesos a través de *todos* los pasos anteriores. Por eso el entrenamiento requiere **derivadas parciales totales** y no simples.

3.2 Función de costo

Para el problema de estabilización/posicionamiento en el estado deseado $\mathbf{x}^*$:

$$J = \frac{1}{2} \sum_{k=1}^N (\mathbf{x}_k - \mathbf{x}^*)^T Q (\mathbf{x}_k - \mathbf{x}^*) \quad (9)$$

con Q matriz de ponderación positiva (en el caso más simple $Q = I$). Los pesos se actualizan por gradiente descendente con derivadas *totales*:

$$w \leftarrow w - \eta \frac{\partial J}{\partial w}, \quad v \leftarrow v - \eta \frac{\partial J}{\partial v} \quad (\text{ídem para } c \text{ y } a)$$

Derivando (9), para cualquier parámetro $\theta \in \{v, c, a, w\}$:

$$\frac{\partial J}{\partial \theta} = \sum_{k=1}^N (\mathbf{x}_k - \mathbf{x}^*)^T Q \frac{\partial \mathbf{x}_k}{\partial \theta} \quad (10)$$

El problema se reduce entonces a calcular las derivadas totales del estado $\frac{\partial \mathbf{x}_k}{\partial \theta}$.

3.3 Derivación de la ecuación recursiva del DBP

Sustituyendo el controlador en el modelo, el estado en $k+1$ es

$$\mathbf{x}_{k+1} = \Phi(\mathbf{x}_k, N(\mathbf{x}_k; \theta))$$

Aplicando la regla de la cadena, y teniendo en cuenta que θ afecta a $\mathbf{x}_{k+1}$ por *dos* caminos —(i) directamente a través de u_k en el paso presente, y (ii) indirectamente a través de $\mathbf{x}_k$, que ya depende de θ por los pasos anteriores—:

$$\frac{\partial \mathbf{x}_{k+1}}{\partial \theta} = \frac{\partial \Phi}{\partial u_k} \frac{\partial u_k}{\partial \theta} + \frac{\partial \Phi}{\partial \mathbf{x}_k} \frac{\partial \mathbf{x}_k}{\partial \theta} \quad \text{con} \quad \frac{\partial u_k}{\partial \theta} = \frac{\partial u_k}{\partial \theta} + \frac{\partial u_k}{\partial \mathbf{x}_k} \frac{\partial \mathbf{x}_k}{\partial \theta}$$

Combinando ambas expresiones se obtiene la **ecuación recursiva del algoritmo DBP**:

$$\boxed{\frac{\partial \mathbf{x}_{k+1}}{\partial \theta} = \underbrace{\frac{\partial \mathbf{x}_{k+1}}{\partial u_k} \frac{\partial u_k}{\partial \theta}}_{\text{aporte del paso presente}} + \underbrace{\left[\frac{\partial \mathbf{x}_{k+1}}{\partial \mathbf{x}_k} + \frac{\partial \mathbf{x}_{k+1}}{\partial u_k} \frac{\partial u_k}{\partial \mathbf{x}_k} \right]}_{\text{Jacobiano del lazo cerrado } J_t} \frac{\partial \mathbf{x}_k}{\partial \theta}} \quad (11)$$

con condición inicial $\frac{\partial \mathbf{x}_0}{\partial \theta} = 0$ (el estado inicial no depende de los pesos). La recursión avanza *hacia adelante* en el tiempo junto con la simulación del lazo cerrado, acumulando en cada paso el gradiente (10); ésta es la ventaja del DBP frente al BPTT en requerimientos de memoria.

3.4 ¿Qué término se calcula con qué?

Término	Se calcula con...
$\frac{\partial \mathbf{x}_{k+1}}{\partial \mathbf{x}_k}$ (Jacobiano del sistema)	el modelo del sistema (ecuaciones de movimiento)
$\frac{\partial \mathbf{x}_{k+1}}{\partial u_k}$	el modelo del sistema
$\frac{\partial u_k}{\partial \theta}$ para $\theta = w, v, c, a$ (der. simples)	el neuro-controlador (sus ecuaciones son conocidas)
$\frac{\partial u_k}{\partial \mathbf{x}_k}$ (sensibilidad del control al estado)	el neuro-controlador

3.5 Matrices para el robot móvil tipo carro

Discretización. Las ecuaciones están en tiempo continuo; con el método de Euler, paso de muestreo Δt y avance por paso $\Delta r = v \Delta t$ (Clase 10):

$$x_{k+1} = x_k + \Delta r \cos \phi_k, \quad y_{k+1} = y_k + \Delta r \sin \phi_k, \quad \phi_{k+1} = \phi_k - \frac{\Delta r}{L} u_k$$

Reducción del estado. Para la posición final deseada $y^* = 0$, $\phi^* = 0$ (llegar a la recta $y = 0$ horizontal y avanzar por ella), la señal de control no requiere la coordenada x : al converger, $\dot{x} = v$ y x se comporta como el tiempo. El estado usado para el entrenamiento es

$$\mathbf{x}_k = \begin{bmatrix} y_k \\ \phi_k \end{bmatrix}, \quad \mathbf{x}_{k+1} = \Phi(\mathbf{x}_k, u_k) = \begin{bmatrix} y_k + \Delta r \sin \phi_k \\ \phi_k - \frac{\Delta r}{L} u_k \end{bmatrix}, \quad \mathbf{x}^* = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Matrices del modelo (los dos términos de la ecuación (11) que aporta el modelo):

$$\frac{\partial \mathbf{x}_{k+1}}{\partial \mathbf{x}_k} = \begin{bmatrix} 1 & \Delta r \cos \phi_k \\ 0 & 1 \end{bmatrix} \quad (= \text{jacob}), \quad \frac{\partial \mathbf{x}_{k+1}}{\partial u_k} = \begin{bmatrix} 0 \\ -\frac{\Delta r}{L} \end{bmatrix} \quad (= \text{dxdu}) \quad (12)$$

Términos del neuro-controlador. Para la red de una capa oculta con sigmoide bipolar

$$\mathbf{m} = v^T \mathbf{x}_k, \quad n_j = \frac{2}{1 + e^{-(m_j - c_j)/a_j}} - 1, \quad u_k = w^T \mathbf{n}$$

se tiene, con $D = \text{diag}\left(\frac{1 - n_j^2}{2a_j}\right)$ (derivada de la sigmoide bipolar):

$$\frac{\partial u_k}{\partial \mathbf{x}_k} = w^T D v^T = \begin{bmatrix} \frac{\partial u}{\partial y} & \frac{\partial u}{\partial \phi} \end{bmatrix}, \quad \frac{\partial u_k}{\partial w} = \mathbf{n}, \quad \frac{\partial u_k}{\partial v} = \mathbf{x}_k (w^T D)$$

$$\frac{\partial u_k}{\partial c_j} = w_j \frac{n_j^2 - 1}{2a_j}, \quad \frac{\partial u_k}{\partial a_j} = w_j \frac{(n_j^2 - 1)(m_j - c_j)}{2a_j^2}$$

Ecuación recursiva explícita. El Jacobiano del lazo cerrado es

$$J_t = \frac{\partial \mathbf{x}_{k+1}}{\partial \mathbf{x}_k} + \frac{\partial \mathbf{x}_{k+1}}{\partial u_k} \frac{\partial u_k}{\partial \mathbf{x}_k} = \begin{bmatrix} 1 & \Delta r \cos \phi_k \\ -\frac{\Delta r}{L} \frac{\partial u}{\partial y} & 1 - \frac{\Delta r}{L} \frac{\partial u}{\partial \phi} \end{bmatrix} \quad (= \text{jacob_t})$$

y para cada grupo de parámetros $\theta \in \{w, v, c, a\}$:

$$\frac{\overline{\partial \mathbf{x}_{k+1}}}{\partial \theta} = \begin{bmatrix} 0 \\ -\frac{\Delta r}{L} \end{bmatrix} \frac{\partial u_k}{\partial \theta} + J_t \frac{\overline{\partial \mathbf{x}_k}}{\partial \theta}, \quad \frac{\overline{\partial J}}{\partial \theta} = \sum_k (\mathbf{x}_k - \mathbf{x}^*)^T \frac{\overline{\partial \mathbf{x}_k}}{\partial \theta}$$

Observación (caso de estado completo). Si se conservara también la coordenada x en el estado ($\mathbf{x}_k = [x_k \ y_k \ \phi_k]^T$), las matrices serían

$$\frac{\partial \mathbf{x}_{k+1}}{\partial \mathbf{x}_k} = \begin{bmatrix} 1 & 0 & -\Delta r \sin \phi_k \\ 0 & 1 & \Delta r \cos \phi_k \\ 0 & 0 & 1 \end{bmatrix}, \quad \frac{\partial \mathbf{x}_{k+1}}{\partial u_k} = \begin{bmatrix} 0 \\ 0 \\ -\Delta r/L \end{bmatrix}$$

pero para el objetivo $y^* = 0$, $\phi^* = 0$ el curso usa el estado reducido $[y \ \phi]^T$, que es el implementado en el programa de la Pregunta 5.

Pregunta 4 (4 ptos) — Variantes del entrenamiento

(a) Sistema de dos variables de estado de las cuales sólo se mide una: ¿qué cambios hay que hacer en el entrenamiento? (b) ¿Cómo se desarrolla el aprendizaje incremental para estabilizar un sistema bilineal inestable en un amplio rango de operación? (c) ¿Qué cambios se realizan si se desea que $\mathbf{x}$ converja en $\mathbf{x}^*$?

(a) Realimentación parcial: sólo se mide una de las dos variables de estado

Con medición completa, el neuro-controlador tiene como entradas las dos variables: $u_k = N(x_{1,k}, x_{2,k})$. Si sólo se mide x_1 , los cambios son:

- **Entradas del controlador.** El controlador sólo puede alimentarse con la variable medida. Para no perder la información dinámica de la variable no medida, se agregan como entradas *valores pasados* de la medición (retardos z^{-1}): $u_k = N(x_{1,k}, x_{1,k-1}, \dots)$. Con suficientes retardos, la red reconstruye implícitamente el estado no medido (actúa como un observador implícito), siempre que el sistema sea observable desde x_1 .
- **Función de costo.** El entrenamiento es basado en modelo (el lazo se *simula*), por lo que todas las variables de estado del modelo están disponibles durante el entrenamiento: la función de costo puede seguir penalizando el estado completo, o sólo la variable medida usando la matriz de ponderación, p. ej. $Q = \text{diag}(1, 0)$ en (9).
- **Cambios en las derivadas del DBP.** Como u_k ya no depende de x_2 directamente, la sensibilidad respecto al estado se anula en las componentes no medidas: $\frac{\partial u_k}{\partial \mathbf{x}_k} = \begin{bmatrix} \frac{\partial u}{\partial x_1} & 0 \end{bmatrix}$. Además, al usar entradas retardadas, u_k depende también de $x_{1,k-1}$: la recursión de derivadas totales debe formularse sobre el *estado aumentado* $[\mathbf{x}_k; x_{1,k-1}; \dots]$ que incluye los retardos como estados adicionales del lazo cerrado.
- **Comentario.** El desempeño alcanzable puede degradarse respecto a la realimentación completa y el entrenamiento suele ser más lento y delicado (hay que monitorear la evolución del costo). Sin embargo, tal como se estudió en el curso (programa `DynamicBPControl2a.m`), un neuro-controlador con salida parcial y retardos puede estabilizar incluso sistemas inestables; conviene validar su robustez frente a perturbaciones externas y ruido de medición, pues la medida entra directamente a la red.

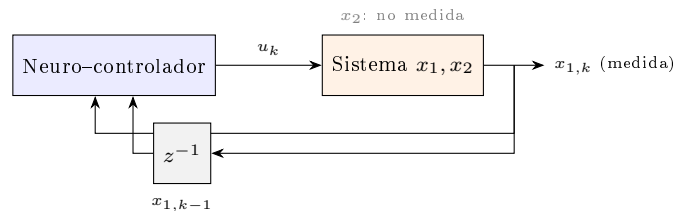


Figura 6: Controlador con realimentación parcial: la variable medida y sus valores pasados como entradas.

(b) Aprendizaje incremental para un sistema bilineal inestable en amplio rango

Al igual que los seres humanos, la red debe entrenarse *empezando con tareas simples* y continuar con tareas más complejas de manera incremental. Para la mayoría de los sistemas, las tareas simples son las cercanas a un estado estable (de equilibrio). El procedimiento:

1. **Etapla inicial (tarea simple).** Se entrena el neuro-controlador con condiciones iniciales *cercanas al estado de equilibrio* $\mathbf{x}^*$ (y, si conviene, horizontes de simulación N cortos). Como el sistema es inestable, empezar cerca del equilibrio evita que las trayectorias diverjan durante las primeras épocas, cuando la red aún no controla, y que los gradientes crezcan descontroladamente.
2. **Ampliación progresiva.** Una vez que el costo converge, se amplía gradualmente el rango de condiciones iniciales (más lejos del equilibrio) y/o el horizonte N , *re-usando los pesos ya entrenados como inicialización* de la nueva etapa (la red no empieza de cero: refina lo aprendido).
3. **Conjunto de posiciones iniciales.** En cada etapa se entrena simultáneamente con un *conjunto* de posiciones iniciales representativas (en los programas del curso, cuatro), acumulando los gradientes de todas las trayectorias antes de actualizar los pesos; así el controlador generaliza a otras posiciones no vistas.
4. **Repetición.** Se repite el proceso hasta cubrir todo el amplio rango de operación requerido, monitoreando que el costo decrezca en cada etapa.

(c) Cambios para que $\mathbf{x}$ converja en $\mathbf{x}^*$ (regulación / tracking)

El neuro-controlador entrenado para estabilización ($\mathbf{x} \rightarrow 0$ con $u \rightarrow 0$), representado por $u_k = N(\mathbf{x}_k)$, se adapta cambiando *apropiadamente sus entradas y su salida*:

$$\boxed{u_k = u^* + N(\mathbf{x}_k - \mathbf{x}^*)} \quad (13)$$

es decir: (i) a las entradas de la red se les resta el estado deseado $\mathbf{x}^*$ (la red ve el *error* $\mathbf{x}_k - \mathbf{x}^*$, que debe converger a cero), y (ii) a la salida de la red se le suma el control de equilibrio u^* . Los valores $\mathbf{x}^*$ y u^* no pueden elegirse de forma independiente: deben cumplir la **condición de equilibrio** del sistema

$$\mathbf{x}^* = f(\mathbf{x}^*, u^*) \quad (14)$$

(para el sistema bilineal de la parte (b), esta condición determina el u^* correspondiente a cada $\mathbf{x}^*$ alcanzable). Si se prefiere, alternativamente puede re-entrenarse la red directamente con la función de costo $J = \frac{1}{2} \sum_k (\mathbf{x}_k - \mathbf{x}^*)^T (\mathbf{x}_k - \mathbf{x}^*)$, con `out_des` = $\mathbf{x}^*$, pero la adaptación de entradas/salida evita el re-entrenamiento y permite cambiar $\mathbf{x}^*$ en línea.

Pregunta 5 (4 ptos) — Explicación del programa Matlab de cálculo y acumulación de derivadas totales (DBP) para el robot tipo carro

Explicar el significado de cada línea del programa. Explicar cómo está implementada la ecuación recursiva del cálculo de derivadas totales según el algoritmo DBP. La posición final deseada es $y^ = 0$, $\phi^* = 0$; la variable $x(1,1)$ corresponde a una de las coordenadas y $x(2,1)$ al ángulo ϕ . Indicar todas las matrices que deben inicializarse antes del bucle `for-end`.*

Según lo visto en la Pregunta 3, $x(1,1) = y_k$ (la coordenada que aparece en el objetivo $y^* = 0$; en el modelo actualizado con $\sin$ de $x(2,1)$) y $x(2,1) = \phi_k$. La red tiene una capa oculta de n_h neuronas sigmoideas bipolares con pesos de entrada v , centros c , pendientes a y pesos de salida w ; $r = \Delta r = v\Delta t$, $L = L$, y j indexa la posición inicial de la trayectoria simulada (bucle externo no mostrado).

```

1 for k = 1:ndata-1
2     in_red = [ x(1,1)
3               x(2,1) ];
4     m = v'*in_red;
5     n = 2.0./(1 + exp(-(m-c)./a)) - 1;
6     out_red = w'*n;
7     u(k,j) = out_red';
8     estado(k,:,j) = x';
9     jacob = [ 1 r*cos(x(2,1))
10              0 1 ];
11     dxdu = [ 0
12              -r/L ];
13     z(1,1) = x(1,1) + r*sin(x(2,1));
14     z(2,1) = x(2,1) - r/L*u(k,j);
15     x = z;
16     dndm = diag((1 - n.*n)./(2*a));
17     dudw_s = [ n ];
18     dudv_s = in_red*w'*dndm;
19     dudc_s = w .* ((n.*n-1)./(2.0.*a));
20     duda_s = w .* ((n.*n-1).*(m-c)./(2*a.*a));
21     dudx = w'*dndm*v';
22     jacob_t = dxdu*dudx + jacob;
23     dx1dw_t = dxdu(1,1).*dudw_s + jacob_t(1,1).*dx1dw_t + jacob_t(1,2).*dx2dw_t;
24     dx2dw_t = dxdu(2,1).*dudw_s + jacob_t(2,1).*dx1dw_t + jacob_t(2,2).*dx2dw_t;
25     dx1dv_t = dxdu(1,1).*dudv_s + jacob_t(1,1).*dx1dv_t + jacob_t(1,2).*dx2dv_t;
26     dx2dv_t = dxdu(2,1).*dudv_s + jacob_t(2,1).*dx1dv_t + jacob_t(2,2).*dx2dv_t;
27     dx1dc_t = dxdu(1,1).*dudc_s + jacob_t(1,1).*dx1dc_t + jacob_t(1,2).*dx2dc_t;
28     dx2dc_t = dxdu(2,1).*dudc_s + jacob_t(2,1).*dx1dc_t + jacob_t(2,2).*dx2dc_t;
29     dx1da_t = dxdu(1,1).*duda_s + jacob_t(1,1).*dx1da_t + jacob_t(1,2).*dx2da_t;
30     dx2da_t = dxdu(2,1).*duda_s + jacob_t(2,1).*dx1da_t + jacob_t(2,2).*dx2da_t;
31     out_des = [ 0 0 ];
32     out_des = out_des';
33     erJ = (x - out_des);
34     dJdw_t = dJdw_t + erJ(1,1).*dx1dw_t + erJ(2,1).*dx2dw_t;
35     dJdv_t = dJdv_t + erJ(1,1).*dx1dv_t + erJ(2,1).*dx2dv_t;
36     dJdc_t = dJdc_t + erJ(1,1).*dx1dc_t + erJ(2,1).*dx2dc_t;
37     dJda_t = dJda_t + erJ(1,1).*dx1da_t + erJ(2,1).*dx2da_t;
38 end

```

5.1 Significado de cada línea

Bloque A — Evaluación del neuro-controlador (líneas 1–7).

- L1: bucle sobre los pasos de tiempo de la trayectoria simulada ($k = 1, \dots, N - 1$).

- L2-3: `in_red` es el vector de entrada de la red = estado actual $[y_k; \phi_k]$.
- L4: `m` = $v^T \mathbf{x}_k$: entradas netas (sumas ponderadas) de las neuronas ocultas.
- L5: `n`: salidas de las neuronas ocultas con sigmoide bipolar $n_j = \frac{2}{1+e^{-(m_j-c_j)/a_j}} - 1$ (rango $(-1,1)$, centro c_j , pendiente a_j).
- L6: `out_red` = $w^T \mathbf{n}$: salida de la red = señal de control u_k .
- L7: se almacena el control aplicado en el paso k de la trayectoria j .
- L8: se almacena la trayectoria del estado (para graficar/validar).

Bloque B — Modelo del sistema y sus derivadas (líneas 9–15).

- L9-10: `jacob` = $\frac{\partial \mathbf{x}_{k+1}}{\partial \mathbf{x}_k} = \begin{bmatrix} 1 & \Delta r \cos \phi_k \\ 0 & 1 \end{bmatrix}$: Jacobiano del modelo discretizado del robot (término que aporta *el modelo del sistema* a la ecuación recursiva).
- L11-12: `dxdu` = $\frac{\partial \mathbf{x}_{k+1}}{\partial u_k} = [0; -\Delta r/L]$: sensibilidad del estado siguiente respecto al control (también del *modelo*).
- L13-14: ecuaciones de movimiento discretizadas: $y_{k+1} = y_k + \Delta r \sin \phi_k$; $\phi_{k+1} = \phi_k - \frac{\Delta r}{L} u_k$.
- L15: se avanza el estado: $\mathbf{x}_k \leftarrow \mathbf{x}_{k+1}$ (simulación del lazo cerrado).

Bloque C — Derivadas simples del neuro-controlador (líneas 16–21).

- L16: `dndm` = $\text{diag}(\frac{1-n_j^2}{2a_j})$: derivada de la sigmoide bipolar respecto a su entrada neta.
- L17: `dudw_s` = $\frac{\partial u_k}{\partial w} = \mathbf{n}$ (derivada *simple*: sufijo `_s`).
- L18: `dudv_s` = $\frac{\partial u_k}{\partial v} = \mathbf{x}_k (w^T D)$, matriz $2 \times n_h$ con componentes $x_i w_j n'_j$.
- L19: `dudc_s` = $\frac{\partial u_k}{\partial c}$: como $\frac{\partial n_j}{\partial c_j} = -\frac{\partial n_j}{\partial m_j}$, resulta $w_j \frac{n_j^2-1}{2a_j}$.
- L20: `duda_s` = $\frac{\partial u_k}{\partial a} = w_j \frac{(n_j^2-1)(m_j-c_j)}{2a_j^2}$.
- L21: `dudx` = $\frac{\partial u_k}{\partial \mathbf{x}_k} = w^T D v^T = [\frac{\partial u}{\partial y} \quad \frac{\partial u}{\partial \phi}]$: sensibilidad del control respecto al estado (del *neuro-controlador*).

Bloque D — Ecuación recursiva del DBP (líneas 22–30).

- L22: `jacob_t` = `dxdu*dudx` + `jacob` = $\frac{\partial \mathbf{x}_{k+1}}{\partial u_k} \frac{\partial u_k}{\partial \mathbf{x}_k} + \frac{\partial \mathbf{x}_{k+1}}{\partial \mathbf{x}_k} = J_t$: **Jacobiano del lazo cerrado** (combina modelo y controlador).
- L23-30: implementan, componente a componente y para cada grupo de parámetros $\theta \in \{w, v, c, a\}$, la ecuación recursiva

$$\overline{\frac{\partial \mathbf{x}_{k+1}}{\partial \theta}} = \frac{\partial \mathbf{x}_{k+1}}{\partial u_k} \frac{\partial u_k}{\partial \theta} + J_t \overline{\frac{\partial \mathbf{x}_k}{\partial \theta}}$$

La fila 1 (variable y): `dx1dtheta_t` = `dxdu(1,1)` · $\frac{\partial u}{\partial \theta}$ + $J_t(1,1)$ `dx1dtheta_t` + $J_t(1,2)$ `dx2dtheta_t`, y análogamente la fila 2 (variable ϕ). En el lado derecho, `dx1dtheta_t` y `dx2dtheta_t` contienen los valores del paso *anterior* ($\overline{\frac{\partial \mathbf{x}_k}{\partial \theta}}$), y tras la asignación quedan actualizadas al paso $k+1$.

Bloque E — Error y acumulación del gradiente del costo (líneas 31–38).

- L31-32: `out_des` = $[y^*; \phi^*] = [0; 0]$: posición final deseada.

- L33: $\text{erJ} = \mathbf{x}_{k+1} - \mathbf{x}^*$: error de la trayectoria en el paso $k+1$ (nótese que $\mathbf{x}$ ya fue actualizado en L15, coherente con que las derivadas totales calculadas corresponden a $\mathbf{x}_{k+1}$).
- L34–37: acumulación del gradiente total del costo $J = \frac{1}{2} \sum_k \|\mathbf{x}_k - \mathbf{x}^*\|^2$:

$$\frac{\partial J}{\partial \theta} += (\mathbf{x}_{k+1} - \mathbf{x}^*)^T \frac{\partial \mathbf{x}_{k+1}}{\partial \theta} = e_1 \text{dx1d}\theta_t + e_2 \text{dx2d}\theta_t$$

Al terminar el bucle, dJdw_t , dJdv_t , dJdc_t , dJda_t contienen los gradientes totales listos para actualizar los pesos por gradiente descendente ($w \leftarrow w - \eta \text{dJdw_t}$, etc., típicamente tras acumular sobre todas las posiciones iniciales j).

5.2 Cómo está implementada la ecuación recursiva del DBP

La ecuación recursiva (11) aparece en el programa en dos partes:

1. **Construcción del Jacobiano de lazo cerrado** (L22): $\text{jacob_t} = \text{dxdu} * \text{dudx} + \text{jacob}$, que corresponde exactamente al término entre corchetes de (11): la matriz 2×2 que propaga las derivadas totales del paso k al paso $k+1$; combina el *modelo* (jacob , dxdu) con el *controlador* (dudx).
2. **Propagación + inyección** (L23–30): cada línea suma (i) el **aporte nuevo** del paso presente, $\frac{\partial \mathbf{x}_{k+1}}{\partial u_k} \frac{\partial u_k}{\partial \theta}$ (producto $\text{dxdu}(\mathbf{i}, 1) \times$ derivada simple $\text{dud}\theta_s$), con (ii) la **propagación** de la historia acumulada, $J_t \frac{\partial \mathbf{x}_k}{\partial \theta}$ (combinación de $\text{dx1d}\theta_t$ y $\text{dx2d}\theta_t$ del paso anterior con los elementos de jacob_t). La recursión se inicializa en cero antes del bucle y avanza *hacia adelante* junto con la simulación: por eso el DBP no necesita almacenar toda la trayectoria como el BPTT.

Observación fina: en rigor, las dos filas de cada grupo deberían actualizarse *simultáneamente* (la línea de dx2dw_t usa dx1dw_t que acaba de ser sobrescrito); una implementación estricta guarda copias temporales de los valores del paso anterior antes de actualizar. El efecto numérico es pequeño si Δr es pequeño (los términos cruzados son de orden Δr), pero es un detalle a cuidar al programar.

5.3 Matrices que deben inicializarse antes del bucle for-end

Con n_h el número de neuronas ocultas:

Variable	Dimensión	Inicialización
$\mathbf{x}$	2×1	posición inicial $[y_0; \phi_0]$ de la trayectoria j
$\mathbf{v}$	$2 \times n_h$	pesos de entrada de la red (aleatorios pequeños)
$\mathbf{c}, \mathbf{a}, \mathbf{w}$	$n_h \times 1$ c/u	centros, pendientes y pesos de salida de la red
$\text{dx1dw_t}, \text{dx2dw_t}$	$n_h \times 1$	ceros (condición inicial $\frac{\partial \mathbf{x}_0}{\partial w} = 0$)
$\text{dx1dv_t}, \text{dx2dv_t}$	$2 \times n_h$	ceros
$\text{dx1dc_t}, \text{dx2dc_t}$	$n_h \times 1$	ceros
$\text{dx1da_t}, \text{dx2da_t}$	$n_h \times 1$	ceros
dJdw_t	$n_h \times 1$	ceros
dJdv_t	$2 \times n_h$	ceros
$\text{dJdc_t}, \text{dJda_t}$	$n_h \times 1$	ceros
$\mathbf{u}, \text{estado}, \mathbf{z}$	—	arreglos de almacenamiento (y $\mathbf{z}$ 2×1)
$\mathbf{r}, \mathbf{L}, \text{ndata}$	escalares	$\Delta r = v\Delta t$, longitud del robot, número de datos

Precisión importante sobre el alcance de la inicialización: las derivadas totales de estado ($\text{dx1d}\theta_t$, $\text{dx2d}\theta_t$) deben ponerse en cero *al inicio de cada trayectoria* (cada posición inicial j),

porque cada trayectoria comienza con un estado inicial que no depende de los pesos. En cambio, los acumuladores del gradiente ($dJdw_t$, $dJdv_t$, $dJdc_t$, $dJda_t$) se ponen en cero *una vez por época de entrenamiento*, antes del bucle sobre las posiciones iniciales, porque acumulan el gradiente sobre todos los pasos k y todas las trayectorias j (aprendizaje tipo batch con varias posiciones iniciales, como se indicó en la Pregunta 4(b)).

Lima, julio de 2026 — Documento de estudio basado íntegramente en el material del curso.